

Step 5 Summary — Neural Networks for Imperfect-Information Games

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

May 2026

Contents

5 Step 5 — Neural Networks for Imperfect-Information Games	1
Why Neural Networks	1
5.1 Part 1 — Fundamentals	2
5.2 Part 2 — Architecture & Composition	4
5.3 Part 3 — Neural Networks in CFR	10
5.4 Part 4 — Other Neural Applications	14
5.5 Part 5 — Experimental / Frontier	14
5.6 Synthesis	14

5 Step 5 — Neural Networks for Imperfect-Information Games

This is a condensed survey of neural-network methods for imperfect-information games — the material that Step 5 widened from its original Deep CFR focus into a broader treatment of how neural networks are built, trained, and deployed in games where players cannot see the full state. It serves two purposes: as a quick refresher while progressing through later steps, and as a primary source for the Step 15 public report synthesis. Step 5 is a deliberate one-off deviation from the usual learning cycle — the from-scratch implementation phase is deferred and the theory is broadened in its place — so this document is a breadth-first map of the field rather than a record of code. Mathematics is kept light: key equations are stated where they sharpen intuition, but derivations and proofs are left to the cited sources.

Why Neural Networks

Tabular CFR and its Monte Carlo variants (Steps 2–4) store a regret value and a strategy value at each information set, so their memory cost is proportional to the number of information sets, which grows with the size of the game. The Step 5 exploration quantified this limit on Leduc poker, a game small enough to hold in a table: tabular MCCFR reached an exploitability of 0.097, whereas Deep CFR under a comparable wall-clock budget reached only 1.70. At this scale the tabular method is preferable; the neural method offers no advantage, because the table already fits.

A neural network replaces the table with a function f_θ that maps a state’s features to a value or an action distribution and generalizes to states never visited during training. This inverts the cost structure: memory becomes independent of the number of information sets, at the price of more expensive iterations and weaker convergence guarantees. The two regimes meet at a crossover in game size — below it, enumeration is cheaper and exact; above it, the table cannot be constructed and function approximation is the only option. The games this research targets lie above the crossover, which is why the remainder of the chapter concerns neural methods.

5.1 Part 1 — Fundamentals

5.1.1 Neural Network Fundamentals

A feed-forward network is a composition of layers, each applying an affine map followed by an elementwise nonlinearity:

$$h = \sigma(Wx + b),$$

where x is the layer input, W and b are the trainable weight matrix and bias vector, and σ is a fixed nonlinearity such as the rectifier $\text{ReLU}(z) = \max(0, z)$. Stacking L such layers gives the network function f_θ , with θ collecting every W and b . The nonlinearities are what allow the composition to represent functions that no single affine map can.

ReLU (rectified linear unit): the standard activation — it passes positive values through unchanged and clamps negatives to zero, which is cheap to compute and avoids the vanishing gradients of older saturating activations.

Training requires a scalar measure of error. For a regression target — for example, a counterfactual value — the mean squared error is standard:

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N (f_\theta(x_i) - y_i)^2,$$

where N is the number of examples in a batch, x_i an input, and y_i its target. When the target is an action distribution, the cross-entropy loss $-\sum_a y_a \log f_\theta(x)_a$ is used instead, with y_a the target probability of action a .

Learning is gradient descent on this loss. Each parameter moves a short distance in the direction that most reduces the loss:

$$\theta \leftarrow \theta - \eta \nabla_\theta L,$$

where η is the learning rate and $\nabla_\theta L$ the gradient of the loss with respect to the parameters. *Backpropagation* computes that gradient: it applies the chain rule layer by layer from the output back to the input,

reusing each layer’s stored intermediate values, so one gradient costs about as much as one forward pass. Automatic differentiation performs this procedure mechanically.

In practice the gradient is estimated from a small random batch rather than the full dataset (*stochastic gradient descent*) — noisier, but far cheaper per step. Momentum reduces that noise by stepping with a running average of gradients, $v \leftarrow \beta v + \nabla_{\theta} L$ and $\theta \leftarrow \theta - \eta v$, where $\beta \in [0, 1)$ sets how strongly past gradients persist. Adam, the default optimizer in the methods of this chapter, additionally divides each parameter’s step by a running estimate of its gradient magnitude, which yields stable progress with little manual tuning of η .

Adam (adaptive moment estimation): combines momentum with per-parameter step sizes derived from running gradient statistics, which is why it trains reliably with little tuning.

A network with sufficient capacity can fit a training batch exactly, including its noise, and then perform poorly on unseen states — *overfitting*. Regularization counters this. Weight decay adds a penalty $\lambda \|\theta\|^2$ to the loss, with λ controlling how strongly large weights are discouraged; dropout randomly zeroes a fraction p of activations during training so the network cannot depend on any single unit; and early stopping halts training once error on held-out data begins to rise.

One reinforcement-learning idea recurs throughout the chapter and is worth stating explicitly: *bootstrapping*. A temporal-difference update estimates a state’s value from the network’s own estimate at the next state rather than from a final outcome:

$$V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)],$$

where s and s' are consecutive states, r the reward, $\gamma \in [0, 1]$ the discount factor, and α the step size; the bracketed quantity is the temporal-difference error, the gap between the current estimate and the bootstrapped target. Bootstrapping makes learning sample-efficient and, as the next section shows, also less stable. The underlying material is in Sutton & Barto (2nd ed.) chapters 9 and 11; any standard deep-learning reference covers backpropagation and optimization in full.

5.1.2 Putting the Fundamentals Together on Leduc Poker

These pieces combine into the simplest neural solver for a game. Take Leduc poker, the testbed from Steps 3–4: its information state — the private card, the betting history, and the pot — encodes as a vector of about thirty numbers. A small network maps that vector to a value for each action; backpropagation with Adam fits it to target values generated during play, and light regularization prevents it from memorizing the noise in those targets. Used well, the network provides two capabilities the table lacks: it generalizes, so a value learned at one information set informs similar unseen ones, and its memory cost is fixed by the number of parameters rather than by the number of information sets.

The same machinery degrades when overdriven, and the exploration made the failure modes concrete. Capacity is the clearest. On Leduc the smallest network tested, two hidden layers of thirty-two units,

reached a lower exploitability than networks several times larger, because at a limited sample budget the larger networks held too many parameters for too little data and fit noise rather than structure. A learning rate set too high destabilizes the descent; excessive regularization flattens the sharp, near-deterministic strategies CFR converges to, since heavy weight decay or dropout pulls the output toward the uniform distribution. The general principle is that capacity and regularization must be matched to the amount and quality of the data rather than maximized by default — a point Part 2 develops into explicit sizing guidance.

5.1.3 Non-Stationarity and the Deadly Triad

Combining the components above introduces instability. Sutton & Barto identify a *deadly triad*: when function approximation, bootstrapping, and off-policy learning are used together, the parameters can diverge rather than converge, because each update’s bootstrapped target depends on the same parameters being modified. Any one component is safe in isolation; the combination is not. The remedies are practical rather than theoretical. A *target network* — a delayed copy of the parameters used only to compute bootstrap targets — and a *replay buffer* that decorrelates consecutive samples are together what make Deep Q-Networks trainable, and both reappear in the neural game solvers of Parts 3 and 4.

Games introduce a second source of instability absent from single-agent learning. In supervised learning the data distribution is fixed, and in single-agent reinforcement learning it stabilizes once the policy converges. In a game the opponent is also adapting, so the distribution of situations a network encounters changes as training proceeds; the function being approximated is itself moving. This non-stationarity is not incidental — it is the central difficulty of the setting this research addresses. Several methods in later parts are best understood as responses to it: self-play against frozen or time-averaged opponents restores a stationary target, population-based methods replace the moving target with an explicit set of fixed opponents, and the opponent-modeling and fast-adaptation techniques of Parts 4 and 5 attempt to track it directly.

5.2 Part 2 — Architecture & Composition

5.2.1 Inductive Bias: Matching Architecture to Game Structure

Every architecture encodes assumptions about the structure of its input before it sees any data; these built-in assumptions are its *inductive bias*. A fully connected network assumes nothing — every input feature may interact with every other — which is general but data-hungry, since the network must learn any structure from scratch. Choosing an architecture whose bias matches the game’s structure reduces the number of parameters and the amount of data needed to reach a given accuracy, and it improves generalization to states absent from training. The architecture decisions in the rest of Part 2 are, at bottom, choices of inductive bias.

Three correspondences cover most cases in imperfect-information games. When the state is spatial — a board or a grid — a convolutional network assumes that the same local pattern is meaningful wherever it appears, so a feature learned in one region transfers to others; this *translation equivariance* is why

convolutions dominate grid domains. When part of the state is an unordered collection — a hand of cards, or the set of other players — the network should be *permutation-invariant*, producing the same output regardless of the order in which those elements are presented, so that it does not expend capacity learning that the order is irrelevant. When the state is a history revealed over time, as under partial observability, a recurrent network or an attention mechanism lets the network summarize the sequence and carry forward what earlier observations implied. Matching these biases to the game is the difference between a network that learns efficiently and one that merely could in principle.

5.2.2 Layer Types in Practice: MLP, CNN, RNN, Attention/Transformer

The families differ in their *wiring* — which units connect to which — and in whether the connecting weights are *shared* across positions or time. Everything else (the neuron, the nonlinearity, backpropagation) is identical. In the figure below, edge colour denotes a distinct weight, so a repeated colour marks a shared weight; the paragraphs that follow read off each panel.

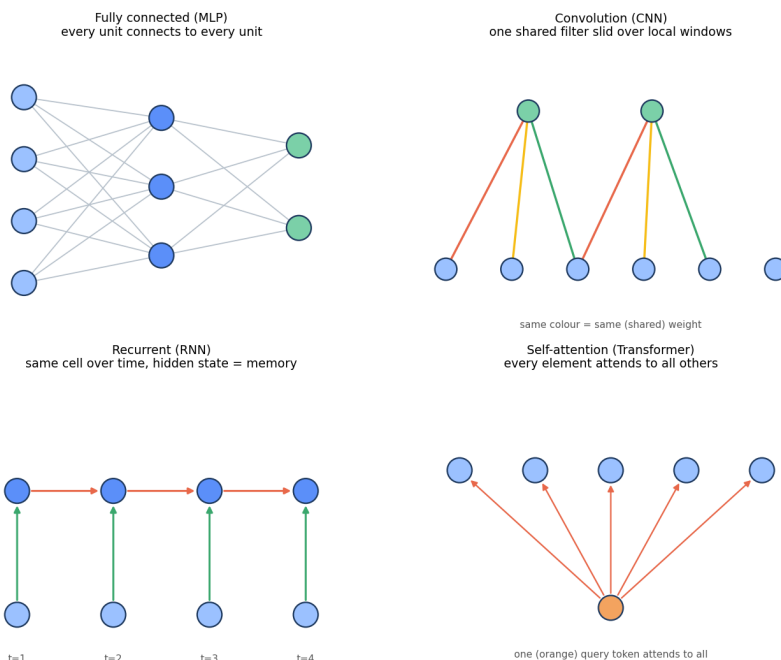


Figure 1: Connectivity of the four layer families: a fully connected MLP, a convolution that reuses one filter across local windows, a recurrent cell reused across time, and self-attention.

A **fully connected layer** — the multilayer perceptron, or MLP — connects every unit to every unit in the next layer, each connection carrying its own weight. It assumes no structure and can represent any relationship in principle, but the number of weights is the product of the two layer sizes, and all structure must be learned from data. It is the right choice for a flat feature vector with no spatial or sequential meaning, such as a poker information state.

A **convolutional layer** (CNN) connects each output unit to only a small local window of the input — its *receptive field* — and applies the same small set of weights, the *filter*, at every position. Two consequences follow from this one change in wiring: the number of weights depends on the filter size rather than the

input size, and a pattern learned at one location is recognized at every location (translation equivariance). Stacking convolutions enlarges the effective receptive field, so later layers combine local patterns into larger ones. This is the standard choice for boards and grids.

Weight sharing: reusing the same weights at many positions (CNN) or time steps (RNN).

It is what makes these layouts parameter-efficient and is the mechanism behind their equivariance.

A **recurrent layer** (RNN) applies the same cell repeatedly along a sequence: at each step it takes the current input together with the previous step’s *hidden state* and produces a new hidden state, which is passed forward. Because the cell is shared across steps, the number of weights is independent of sequence length, and the hidden state acts as a memory of everything seen so far — the property needed when the state is a history revealed over time, as under partial observability. Gated variants (LSTM, GRU) add learned gates that control what to keep and what to discard, which lets them retain information across longer spans.

A **self-attention layer** (the Transformer) lets every element interact with every other directly. Each element emits a *query*, a *key*, and a *value*; the output for an element is a weighted average of all elements’ values, with the weights set by the similarity between its query and each key. Because the operation ignores element order, it is permutation-equivariant, and order must be supplied explicitly through positional encodings when it matters. Attention captures long-range dependencies that a recurrent layer reaches only indirectly, at the cost of computation that grows with the square of the number of elements. It suits variable-length histories and set-valued inputs.

A genuinely unordered input — a hand of cards, or the set of other players — calls for **permutation invariance**, which comes not from a new layer type but from a layout: apply the same small network to each element, then combine the results with an order-independent operation such as a sum, mean, or maximum. This is the recipe behind Deep Sets, and it is the natural encoder for a collection whose order carries no meaning.

In practice a network rarely uses one family alone. The usual design is a pipeline: a separate *encoder* matched to each part of the input, the encoder outputs *fused* by concatenation into a shared *trunk*, and one or more *heads* that read predictions off the trunk — commonly a policy head and a value head. A Deep Recurrent Q-Network (DRQN) composes a convolutional encoder, an LSTM, and an output MLP, the recurrence supplying the memory a single feed-forward pass lacks under partial observability — the natural template for a fog-of-war grid agent. AlphaStar goes further, encoding a set of units with a Transformer, the spatial minimap with a CNN, and scalar statistics with an MLP, then fusing all three and passing them through an LSTM core before its action heads. The practical lesson for the rest of Part 2 is that “which architecture” is usually “which encoder for each input, and how to combine them,” rather than a single choice.

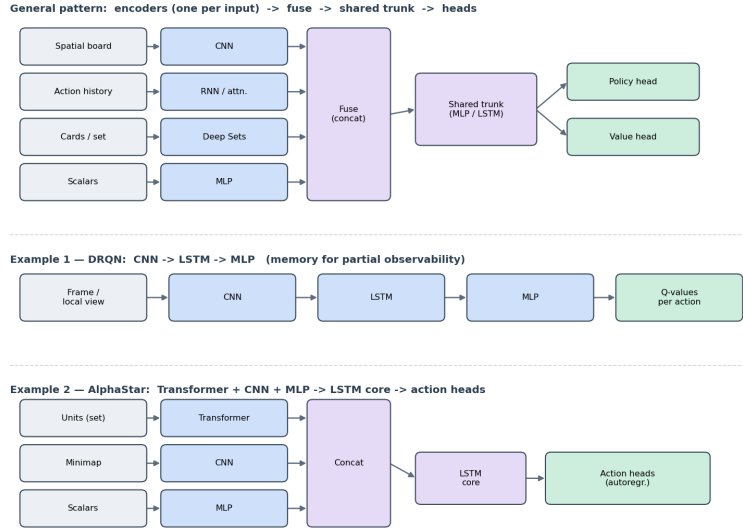


Figure 2: The general encoder / trunk / heads pattern (top); two instances below — DRQN and AlphaStar.

5.2.3 Encoding Game State & History

Before any of these architectures can run, the game state must be turned into numbers. The information state — everything the acting player may legally observe — has to become a fixed-shape tensor, and the way it is laid out determines which architecture can exploit it. Categorical facts are encoded either as *one-hot* vectors — a length- k vector with a single 1 marking which of k categories holds — or, when k is large, as *embeddings*: short learned vectors looked up per category and trained alongside the network. Leduc poker is small enough for one-hot throughout: the private card, the public card, and the betting history each become one-hot fields and the pot a single scalar, producing the roughly thirty-dimensional vector used in Steps 3–4.

One-hot vs. embedding: one-hot is exact but its length grows with the number of categories; an embedding compresses a large vocabulary — every card, or every board tile — into a few learned dimensions and lets similar categories share structure.

Richer inputs call for richer layouts. A spatial state is encoded as a stack of grids, one *channel* per entity type, so that a convolution can read it directly. Pommerman — a grid-based, Bomberman-style game in which agents move around a board placing bombs to destroy walls and trap opponents — is a clean example: its 11x11 board is split into a separate binary plane for each entity type (passages, wooden walls, rigid walls, bombs, flames, power-ups, the acting agent, and each opponent), stacked into an (11, 11, channels) tensor; scalar quantities such as remaining ammunition and blast range are then appended as a short vector or broadcast as constant planes. A history whose order matters is encoded as a *sequence* of per-step vectors for a recurrent layer or a Transformer, rather than flattened into one vector that discards the order. An unordered collection, such as the current set of opponents, is encoded element by element and pooled. In every case the encoding is where the inductive bias of the previous sections is

actually applied: a grid is presented as a grid precisely so that a convolution’s assumptions hold.

One discipline is specific to imperfect-information games and easy to violate: the encoding must contain only what the player can observe, and nothing hidden. If any privileged information — the opponent’s private card, or the true contents of a fogged cell — leaks into the input tensor, the network will learn to use it and effectively see through the fog, yielding a policy that cannot be deployed against a real opponent and an exploitability figure that is meaningless. This is why frameworks such as OpenSpiel define an explicit per-player information-state representation; getting that representation right is a correctness requirement, not a tuning choice.

Two smaller points complete the craft. Continuous features such as the pot or a stack size are normalized to a comparable scale before entering the network, so that no single feature dominates the gradient. And the encoder need not be designed by hand end to end: a network can learn to compress a long history into a compact summary vector — an idea Part 4 develops into learned abstraction and Part 5 into learned belief states.

5.2.4 Sizing & Capacity

A network’s *capacity* — set by its width, the number of units per layer, and its depth, the number of layers — is the size of the family of functions it can represent. Too little capacity and the network *underfits*, unable to express the target strategy; too much and it *overfits*, fitting the noise in a limited sample rather than the underlying structure. The right capacity is the one matched to how much informative data the training procedure actually supplies — which, in a game solver, is often far less than the raw number of states suggests, because the values being fit are themselves noisy estimates.

The Step 5 exploration showed this directly. Running Deep CFR on Leduc under a fixed budget but with different network sizes, the smallest network — two hidden layers of thirty-two units — reached a lower exploitability than networks two and four times wider; the larger networks held more parameters than the few thousand traversals could constrain and spent the surplus fitting sampling noise. The lesson is not that smaller is always better, but that capacity must track the data budget: on a larger game, or with far more traversals, the ranking would reverse.

This argues for sizing a network deliberately rather than defaulting to the largest that fits in memory. A workable recipe is to start small and increase width and depth only while a held-out metric — validation loss, or exploitability for a solver — keeps improving, stopping when it plateaus. Width lets a network represent more distinct patterns at a given level of abstraction; depth lets it compose simple patterns into more abstract ones, which is why spatial and sequential problems reward several layers while a flat feature vector often does not. Capacity should scale with the size and richness of the game, not be maximized by default.

Width need not stay constant across a network. Layers can *shrink* toward the output, compressing the input into a small summary (an encoder, or *funnel*), or *expand* from a small input to reconstruct a structured output (a decoder). Combining the two gives the *hourglass*, whose narrow middle — the *bottleneck* — forces all information through a low-dimensional code; because the network must act or

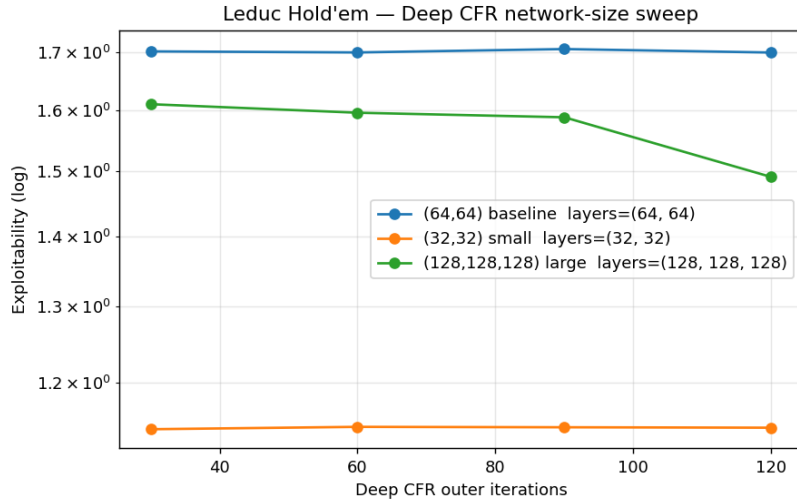


Figure 3: Deep CFR exploitability on Leduc across network sizes (32x32, 64x64, 128x128x128).

reconstruct from that code, it keeps only what matters, which regularizes the model and yields a compact, reusable representation — the seed of a learned abstraction or belief state (Parts 4 and 5). The same pinch dropped into an otherwise uniform stack is a *collar*. The figure below sketches these profiles.

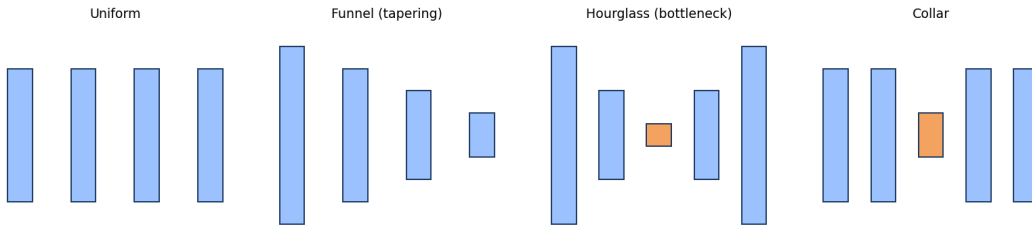


Figure 4: Common network shapes: a uniform stack, a tapering funnel, an hourglass with a central bottleneck (orange), and a uniform stack with a single narrow collar (orange). Bar height represents layer width.

The bottleneck constrains *width*; *skip (residual) connections* address *depth*, adding a layer's input to its output so gradients reach deep layers and very deep networks become trainable — the idea behind ResNets and the deep value networks of Part 4. Both complement the dropout, weight decay, and normalization mentioned earlier: each shapes how capacity is used, not just how much there is.

Skip (residual) connection: a shortcut that adds a layer's input to its output, so the layer only has to learn a correction; it keeps gradients strong through deep networks and is what makes very deep models trainable.

5.2.5 Training Stability & Diagnostics

Two stabilizers from the deadly-triad discussion — the target network and the replay buffer — do most of the work of keeping value-based training from diverging. Two more recur specifically in the game solvers ahead. *Reservoir sampling* maintains a fixed-size, uniformly random sample of all data seen so far, which

matters when a quantity must be averaged over the entire history of training rather than over recent data alone; Deep CFR uses it so that its strategy network trains on an unbiased sample across all iterations (Part 3). *Variance reduction* — subtracting a baseline from sampled returns, or using control variates — lowers the noise that sampling injects into the gradient, so the network converges from fewer samples. Input normalization and sensible weight initialization, mentioned earlier, complete the list: both keep activations and gradients in a usable range from the first step.

Reservoir sampling: a method that keeps a fixed-size sample in which every item seen so far is equally likely to be retained, so a running average stays unbiased without storing all the data.

Even with these in place, a neural solver fails quietly more often than loudly, so reading the diagnostics matters. A loss that falls steadily while the evaluation metric — exploitability, for a solver — stalls is the signature of a network that is *alive but not converged*: training is working, but the budget is too small for the average strategy to settle, which is what the Step 5 networks showed on Leduc at a few thousand traversals. The opposite and more dangerous case is training that is not running at all while appearing to: in the exploration, a one-character bug in an OpenSpiel routine left the advantage networks untrained, yet the program ran to completion and produced plausible output, with exploitability simply frozen at the random-strategy level. The lesson is to confirm that the loss actually moves and that the evaluation metric responds before trusting any result — a silent no-op is indistinguishable from a hard problem unless the curves are checked.

5.3 Part 3 — Neural Networks in CFR

5.3.1 From Tabular CFR to Function Approximation

Tabular CFR keeps two numbers at every information set: a cumulative counterfactual regret for each action, from which the current strategy is derived by regret matching, and a cumulative strategy, whose running average is the Nash approximation the algorithm ultimately returns (Steps 2–4). Both tables grow with the number of information sets — the wall described at the start of this chapter.

Replacing them with neural networks is conceptually simple: a function approximator takes an information-state tensor as input and predicts what the table would have stored. One network can learn the regrets — equivalently, the counterfactual advantages — that determine the current strategy; another can learn the average strategy directly. Because a network generalizes across similar information states, it never has to enumerate them, and its size is fixed by its parameters rather than by the game. The methods below differ mainly in *which* of the two tables they approximate and *how* they generate the data to train it: Deep CFR learns advantages from sampled tree traversals, while NFSP learns the average strategy from self-play without traversing the tree at all.

5.3.2 Deep CFR and Its Single-Network Variants

Deep CFR (Brown et al., 2019) is the direct neural translation of MCCFR. On each iteration it traverses the game tree by external sampling, and at every information set it visits it computes the sampled

counterfactual advantage of each action — how much better that action did than the current strategy on average. These (information-state, advantage) pairs go into a reservoir buffer, and an *advantage network*, one per player, is trained to predict them; the current strategy at any information set is then read off the network’s predicted advantages by regret matching, exactly as the table would have been used. A separate *strategy network*, trained from its own reservoir buffer, learns the average strategy across all iterations — the Nash approximation returned at the end.

The one equation that distinguishes Deep CFR from tabular MCCFR is the advantage network’s objective: a mean squared error between its prediction and the sampled advantages,

$$L(\theta) = \frac{1}{|B|} \sum_{(I, \tilde{a}) \in B} \|f_{\theta}(I) - \tilde{a}\|^2,$$

where B is a batch drawn from the reservoir, I an information-state tensor, $\tilde{a}$ the vector of sampled advantages for its actions, and f_{θ} the advantage network (in practice the samples are weighted by iteration, mirroring linear CFR). Everything else — the traversal, the regret-matching step, the averaging — carries over unchanged from the tabular algorithm.

Two refinements cut the cost. *Single Deep CFR* drops the separate strategy network and recovers the average policy directly from the stored advantage models, halving the networks to train. *DREAM* replaces external sampling with outcome sampling — one trajectory per iteration — and adds a learned baseline as a control variate to absorb the extra variance that outcome sampling brings, the same variance-reduction idea from Part 2. On large poker games Single Deep CFR matches or slightly outperforms Deep CFR while training a single network instead of two, as the original paper’s comparison shows.

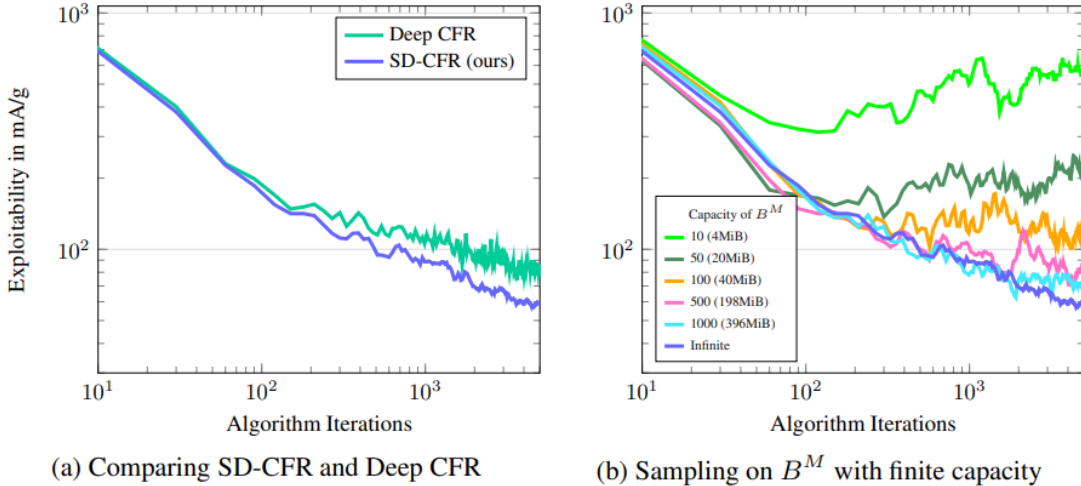


Figure 5: Single Deep CFR (SD-CFR) matches or slightly beats Deep CFR while training only one network (left); the right panel shows how reservoir-buffer capacity affects convergence. From the Single Deep CFR paper — not an experiment from this work.

On Leduc the exploration confirmed both the promise and the catch. Deep CFR trained correctly — its

advantage losses fell as expected — but at 120 iterations it reached an exploitability near 1.70, against 0.097 for tabular MCCFR at comparable wall-clock time. This is not a defect: Leduc is small enough that the table wins outright, and Deep CFR needs several times more iterations to approach Nash. Its advantage appears only when the game is too large for the table to exist — the regime the closing section of this part makes precise.

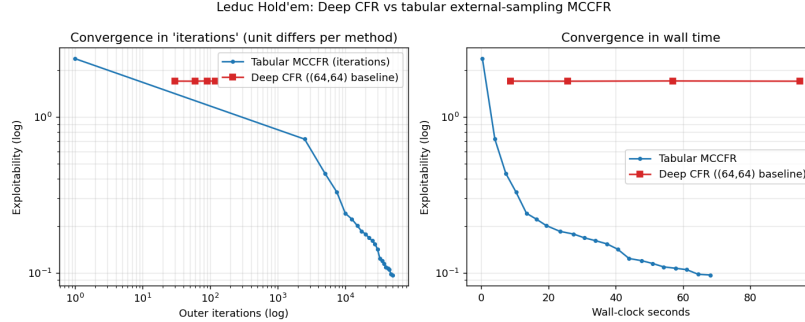


Figure 6: Deep CFR vs. tabular MCCFR exploitability on Leduc.

5.3.3 NFSP — Neural Fictitious Self-Play

NFSP — Neural Fictitious Self-Play (Heinrich & Silver, 2016) — reaches an approximate Nash equilibrium from the reinforcement-learning side, without ever traversing the game tree. Each player keeps two networks. A *best-response network*, a DQN, learns the greedy best response to the opponent’s current behaviour; an *average-policy network*, trained by supervised learning on the player’s own past actions, learns the time-average of those best responses. The average policy is what converges toward Nash — the same averaging that CFR performs, reached through play rather than regret.

The two networks are tied together by the *anticipatory parameter* η : at each step the agent acts from its best-response network with probability η and from its average-policy network otherwise. The fraction of best-response play keeps generating fresh, on-distribution data for the average policy to absorb, while the average policy supplies the stable opponent against which the best response is computed. No reservoir of tree traversals is required — the data comes entirely from episodes of self-play.

The price is sample efficiency. On Leduc the exploration ran NFSP for tens of thousands of episodes and saw exploitability drift around 2.5 rather than fall, because reaching a usable equilibrium even on this small game needs on the order of a million episodes or more — OpenSpiel’s own example uses far more still. NFSP scales to large games, where its model-free simplicity is an asset, but on a teaching benchmark it converges far more slowly than either tabular CFR or Deep CFR.

5.3.4 Trade-offs: When Neural CFR Pays Off

Stepping back: what do neural networks actually buy in CFR, and at what cost? The benefit is generalization and bounded memory — a network shares structure across similar information states and stores a strategy in a fixed number of parameters, so equilibrium computation no longer requires enumerating, or even visiting, every information set. The cost is steep per-iteration overhead and weaker guarantees:

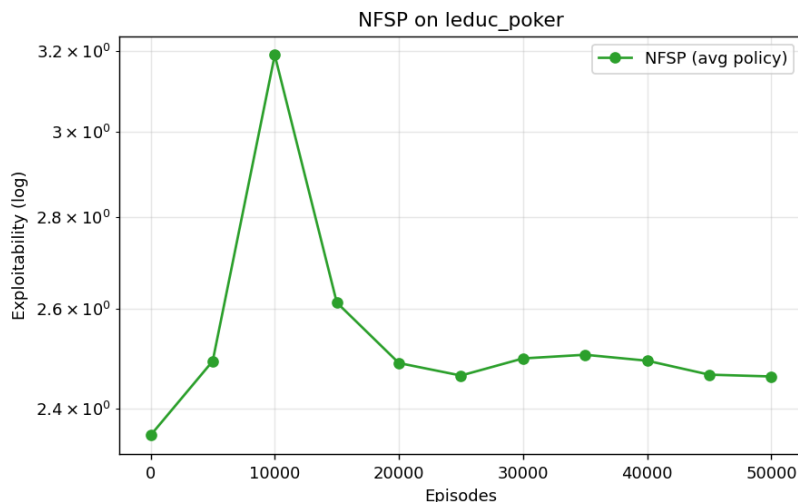


Figure 7: NFSP exploitability on Leduc over training episodes.

each iteration trains networks rather than incrementing counters, convergence is noisier, and the clean monotonic behaviour of tabular CFR becomes approximate. The exploration’s Leduc numbers are the small-game face of this trade-off: where the table fits, it wins.

The trade-off reverses with scale. As a game grows, the tabular tables eventually do not fit in memory at all, while a network’s footprint is unchanged; beyond that crossover the neural method is not merely competitive but the only option. This is why Deep CFR and its relatives matter for full-scale poker even though they lose on Leduc.

One structural caveat bounds where regret-based neural methods apply. CFR computes counterfactual values by reasoning about the whole subtree below an information set, which is cheap when the tree is *wide but shallow* — poker has an enormous number of hands but ends after a few betting rounds. In games that are *deep* — hundreds of sequential moves, as in a gridworld — that reasoning becomes prohibitively expensive, and the model-free self-play methods of Part 4 become the more natural fit. The regret-based family keeps growing within its niche — NeuRD, for instance, recasts the regret update as a policy-gradient step to combine CFR’s stability with neural policy optimization — but its sweet spot remains wide, shallow, imperfect-information games.

5.4 Part 4 — Other Neural Applications

5.4.1 The Four SOTA Algorithm Families

5.4.2 Population-Based: PSRO, NFSP, XFP

5.4.3 Search + RL: ReBeL & Player of Games

5.4.4 Model-Free Self-Play: PPO, DQN, MAPPO, QMIX

5.4.5 NN for Abstraction, Opponent Modeling & Belief Representation

5.4.6 Evaluating Neural Policies: Exploitability at Scale

5.4.7 Generalizing Beyond Poker: Environments & Tooling

5.5 Part 5 — Experimental / Frontier

5.5.1 Memory & Time

5.5.2 Latent Belief Models

5.5.3 Fast Adaptation

5.5.4 Modeling Other Minds & Relations

5.5.5 Uncertainty & Symmetry

5.5.6 Deliberately Out of Scope

5.6 Synthesis

5.6.1 A Decision Map: Which NN Method for Which Game

5.6.2 Connections & Forward Pointers